allgather/ring

An AgentMPI collective scaling analysis

Abstract

The ring allgather passes one block around a circle: in each of $p-1$ rounds every rank forwards the block it received to its right-hand neighbour, so after $p-1$ rounds everyone holds everything. It is the bandwidth-optimal, latency-pessimal member of the allgather family, and across 21 process counts from 2 to 32 it does exactly what its closed form says — $p(p-1)$ messages in $p-1$ rounds, matching at every measured size, with the self-report matching the logged traffic at every size too. There is no remainder path to go wrong: the ring is modular arithmetic and any p is as good as any other. The result the family view delivers is not about correctness but about dominance. In MPI, ring's justification is that it moves the information-theoretic minimum volume, $p(p-1)n$, in messages of constant size n; in AgentMPI Bruck moves the same $p(p-1)n$ in blocks while using $\lceil \log_2 p \rceil$ rounds instead of $p-1$, so ring's classical advantage evaporates and nothing takes its place. At $p = 32$ ring sends 992 messages in 31 rounds and puts 11,904 tokens on the wire against Bruck's 160 messages, 5 rounds and 3,360 tokens, and it takes 1.131 s against 0.225 s. The one axis on which ring still wins is the one no closed form in `cost.py` reports: its messages are 12 tokens each at *every* p, while Bruck's largest grows to 53 and recursive doubling's to 106. If a rank's binding constraint is the size of a single incoming artefact rather than the number of rounds, ring is the only allgather that does not get worse as the population grows. That is a narrow case, and outside it ring should not be chosen.

1 What this family is

The `allgather` collective under the `ring` algorithm, measured at 21 process counts from 2 to 32. Every run is agent-free and deterministic, so the measurements are of the protocol itself.

2 What the analysis notices

- **[ok]** All 21 measured sizes logged exactly the number of messages the closed form predicts.
- **[note]** Wall times span 0.013–1.151 s with no agent in the loop, so these measure the fabric and the protocol rather than model latency.

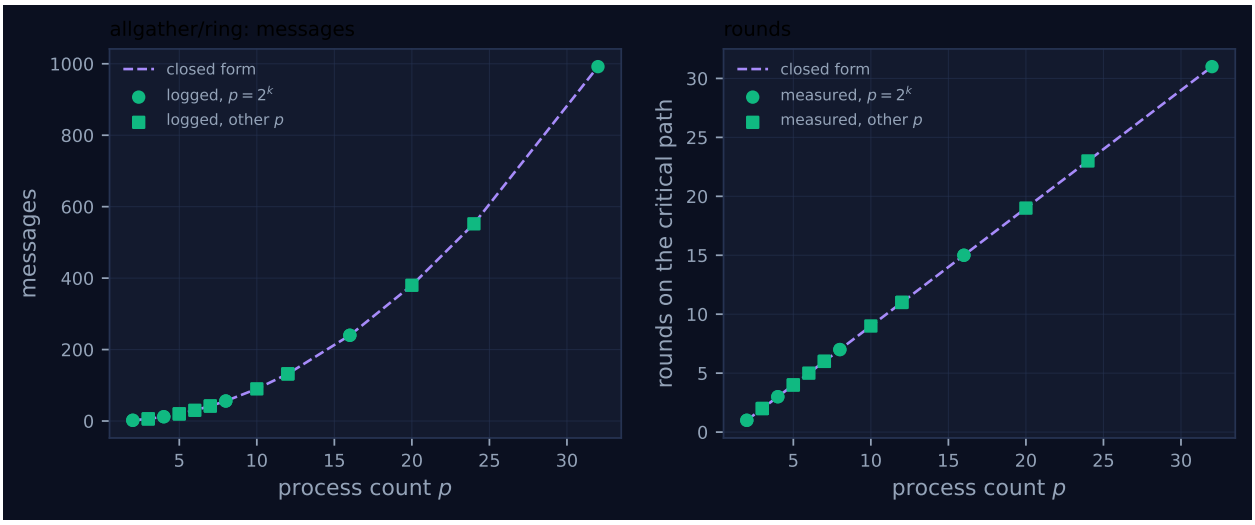

Figure 1: Measured message count and critical-path rounds against process count, with the closed-form prediction. Powers of two are drawn as circles and other sizes as squares, because algorithms take a different code path at sizes that are not powers of two. A red cross marks a size where the collective's own reported count disagrees with the traffic the fabric logged.

3 Every measured size

p	campaign	logged	pred.	self-rep.	rounds	pred.	wall (s)	model	acct.
2	`mb-free`	2	2	2	1	1	0.015	✓	✓
2	`mb-smoke`	2	2	2	1	1	0.013	✓	✓
3	`mb-free`	6	6	6	2	2	0.015	✓	✓
3	`mb-smoke`	6	6	6	2	2	0.016	✓	✓
4	`mb-free`	12	12	12	3	3	0.027	✓	✓
4	`mb-smoke`	12	12	12	3	3	0.025	✓	✓
5	`mb-free`	20	20	20	4	4	0.038	✓	✓
5	`mb-smoke`	20	20	20	4	4	0.042	✓	✓
6	`mb-free`	30	30	30	5	5	0.086	✓	✓
7	`mb-free`	42	42	42	6	6	0.041	✓	✓
7	`mb-smoke`	42	42	42	6	6	0.054	✓	✓
8	`mb-free`	56	56	56	7	7	0.088	✓	✓
8	`mb-smoke`	56	56	56	7	7	0.113	✓	✓
10	`mb-free`	90	90	90	9	9	0.104	✓	✓
12	`mb-free`	132	132	132	11	11	0.219	✓	✓
12	`mb-smoke`	132	132	132	11	11	0.122	✓	✓
16	`mb-free`	240	240	240	15	15	0.196	✓	✓
16	`mb-smoke`	240	240	240	15	15	0.251	✓	✓
20	`mb-free`	380	380	380	19	19	0.369	✓	✓
24	`mb-free`	552	552	552	23	23	0.668	✓	✓
32	`mb-free`	992	992	992	31	31	1.151	✓	✓

4 How the algorithm scales

4.1 Where the cost comes from

The implementation is a nine-line loop. Rank r starts holding its own contribution and, for $p-1$ rounds, executes one `sendrecv` to $(r+1) \bmod p$ and from $(r-1) \bmod p$ carrying a single `{"idx", "val"}` pair, then files what arrived into its output slot by the index it was told. The block that leaves rank r in round k is the block that arrived in round $k-1$, so each of the p blocks travels the whole circle once.

Every quantity follows from that. Each rank sends once per round, so the total is $p(p-1)$ messages; the rounds are strictly sequential, because round k's send is round $k-1$'s receive, so the critical path is $p-1$ deep; and each message carries one block, so the total volume is $p(p-1)n$. That last figure is the information-theoretic floor for an allgather — every rank must *receive* $(p-1)n$ tokens and there are p of them — which is the whole classical case for the algorithm. The `cost.FORMULAS` entry, $(p-1,\ p(p-1),\ p(p-1)n,\ 0)$, is that derivation with a zero fold depth because no operator is applied.

4.2 What the figure shows

Both panels of Figure 1 are smooth, and that is the algorithm's signature. The messages panel is the quadratic $p(p-1)$: 2 at $p=2$, 56 at $p=8$, 240 at $p=16$, 992 at $p=32$, with the measured points on the line at all 21 sizes. The rounds panel is the straight line $p-1$, from 1 to 31. Neither panel has a step, a kink, or a cliff, because neither expression contains a logarithm.

There are no red crosses. The self-reported count matches the logged traffic at all 21 sizes, which is what one expects from an algorithm whose accounting is a single `tr.sent()` inside a single loop with no branches.

The absence of structure is itself the comparison. The sibling families' round panels are staircases that flatten out — Bruck reaches 5 rounds at $p=32$ and stays there for the whole bracket up to 32 — whereas ring's rises without bound. Between $p=16$ and $p=32$ Bruck adds one round and ring adds sixteen.

4.3 Two campaigns, and one visible cost of not measuring latency

Eight sizes were measured twice, once in `mb-free` and once in `mb-smoke`, and the message and round counts are identical in every pair. The wall times are not: at $p=12$ the two runs took 0.219 s and 0.122 s, and at $p=16$, 0.196 s and 0.251 s. That spread — up to 1.8× at the same p — is the size of the noise on the seconds column, and it is worth noting before the timing comparisons in §3, because it means the 5× ring/Bruck wall-clock gap at $p=32$ is well outside the noise while smaller gaps are not.

5 Powers of two and everything else

Nothing distinguishes them, and for once the reason is not that the algorithm handles the remainder gracefully but that it has no notion of a remainder at all. The ring is built from $(r \pm 1) \bmod p$; there is no bit test, no power-of-two participant set, no renumbering, and therefore no second code

path. The circle-marked and square-marked points in Figure 1 lie on the same line for the same reason a clock face works at 12 as well as at 16.

This is the sharpest available contrast with `allreduce/recursive_doubling`, whose one genuine defect lives entirely inside a remainder pre-phase that exists only because a hypercube exchange needs p to be a power of two. Complexity at awkward sizes is where implementation defects live, and ring has none of either.

It is also the contrast with the third allgather algorithm. `allgather/recursive_doubling` cannot run at a non-power-of-two p — the implementation silently rewrites the caller's request to `bruck` — so its family has 9 measured sizes against ring's 21. Ring is the only allgather algorithm in the sweep that is both applicable at every size and free of any size-dependent branch. That is a real property; it is just not enough to make it the right choice.

6 When to choose this algorithm

6.1 Bruck dominates it on every axis a closed form reports

The three allgather algorithms were measured on the same payload, so their logged figures compare directly. Taking `mb-free` at the sizes where all three ran:

	messages			rounds			wire tokens			collective wall (s)	
p	ring	bruck	rec. dbl.	ring	bruck	rec. dbl.	ring	bruck	rec. dbl.	ring	bruck
4	12	8	8	3	2	2	144	44	84	0.024	0.018
8	56	24	24	7	3	3	672	200	384	0.073	0.031
16	240	64	64	15	4	4	2880	832	1616	0.186	0.134
32	992	160	160	31	5	5	11904	3360	6624	1.131	0.225

Bruck sends 6.2× fewer messages, uses 6.2× fewer rounds and puts 3.5× fewer tokens on the wire at $p = 32$, and takes a fifth of the time. There is no size in the sweep at which ring is ahead on any of these four columns; at $p = 2$ and $p = 3$ the messages and rounds tie and ring is still 3× worse on wire volume (72 tokens against 24 at $p = 3$).

The wire-volume result deserves a caveat, because it is an artefact of the benchmark's payload amplified into something that looks structural. Both algorithms move the same $p(p-1)$ blocks; what differs is framing. Ring's message is `{"idx": i, "val": v}` and measures 12 tokens at every p, of which 11 are envelope when v is the two-character string this benchmark uses. Bruck's message is a bare list of $\min(2^k, p - 2^k)$ blocks, so it amortises one envelope over up to $p/2$ blocks. With a realistic 1200-token artefact per block, ring's overhead would be $11 \cdot p(p-1)$ tokens against a payload of $1200 \cdot p(p-1)$ — under 1% — and the two algorithms would converge on volume. `cost.predict` models it that way and gives both \$3.5712 at $p = 32$, $n = 1200$. So the honest statement is that Bruck dominates ring on *messages and rounds* at every size, and on volume only at payloads small enough for framing to matter.

The round count is the one that will not converge. `cost.predict` charges each allgather round `fabric_s` + $n\cdot$ `beta_in` — there is no α term, because an allgather invokes no agent, it only relays artefacts — which at $n = 1200$ is 0.153 s per round and gives 4.7 s for ring against 0.77 s for Bruck at $p = 32$. That 6× gap is linear in $p/\log_2 p$ and grows.

6.2 The one thing ring has, and it is not in any closed form

Ring's messages are 12 tokens at $p = 2$ and 12 tokens at $p = 32$. Bruck's largest message grows as $\lceil p/2 \rceil$ blocks — 4 tokens at $p = 2$, 14 at $p = 8$, 53 at $p = 32$ — and recursive doubling's grows to 106. Scaling the block up to a realistic artefact, at $p = 32$ Bruck's final round delivers 16 blocks in one message; if a block is a 1200-token unit that is a single 19,200-token arrival, against ring's 1200.

That is the case for ring in an agent setting, and it is a real one, because the binding resource on an agent rank is not its port count but its context budget. A rank with a 128k-token budget takes Bruck's 19,200-token message without difficulty at $p = 32$; at $p = 256$ the same arithmetic gives 153,600 tokens in one message and the receive fails. Ring's arrival size is $n + 11$ regardless of p, so a harness whose per-message ceiling is the constraint — large artefacts, many ranks, a strict `strict_context` policy — has exactly one allgather available to it.

Two qualifications. First, nothing in this sweep tests it: the benchmark's contribution is the string `"r7"`, `eager_limit` is set to 10^9 and `strict_context` to `False`, so no run here comes near a budget. The claim is read off the per-round message sizes in the log and the arithmetic of block counts, not measured. Second, the fourth algorithm in the family, `gather_bcast`, is the other way to bound the arrival size, and it is not in this sweep at all — no `coll-allgather-gather_bcast-*` runs exist — so the comparison that would settle which of the two is the better fallback cannot be made from the archive.

6.3 Why it is nevertheless the default at $p \leq 3$, and whether that is right

`allgather` chooses `"ring" if p <= 3 else "bruck"`. At $p = 2$ and $p = 3$ the two algorithms send the same number of messages in the same number of rounds, so the default costs nothing in either quantity the cost model tracks. It does cost wire volume: 24 tokens against 8 at $p = 2$, 72 against 24 at $p = 3$, because of ring's per-message index tagging. The absolute difference is 48 tokens, which is nothing, and Bruck at these sizes performs a final local rotation that ring does not need, so the choice is defensible as "the simpler code where the counts tie". It is worth recording only because it is the one place in the family where the default is not the cheapest option on every axis, and a reader comparing the two families at $p = 3$ will see the discrepancy in the token columns.

7 Threats to this reading

No agents, so nothing here bears on agent behaviour. Every member records `executors: {none: p}`, zero agent calls, zero tokens in and out, $0.0000; the concurrency block reports `total_busy_s: 0` and `idle_fraction: 1.0` at every size. That is a property of `bench_collectives`, whose `rank_main` calls `comm.allgather(f"r{rank}")` and returns, and it is why the viewer's timeline for `mb-free__coll-allgather-ring-32` shows 1,984 green message ticks and not one blue work bar. The message and round counts are exact measurements of protocol structure; there is no measurement of latency here in any sense a harness author cares about.

The wall-clock column is dominated by a polling schedule, and it flatters this algorithm's rivals for a reason that has nothing to do with them. A blocked receive in `comm.py` sleeps `POLL_INTERVAL` = 10 ms and multiplies by 1.4 up to `MAX_POLL_INTERVAL` = 250 ms, so a rank whose neighbour is late learns of the arrival up to a quarter of a second after it happened. An algorithm with a $p - 1$-deep critical path accumulates that penalty $p - 1$ times, and ring's 1.131 s at $p = 32$ is mostly backoff, not work — the effect is visible in extreme form in the `scan/chain` family, where

31 messages take 2.234 s. So the 5× ring/Bruck wall-clock gap at $p = 32$ overstates the structural 6.2× round-count gap in one sense and understates it in another: with a real per-round cost the gap would be cleaner, not smaller. Either way the seconds are not a property of AgentMPI's design and should not be quoted as one.

The volume comparison is a comparison of framing at $n = 1$. Ring's 11,904 tokens against Bruck's 3,360 at $p = 32$ is a real measurement of what crossed the fabric, but 11 of the 12 tokens in a ring message are the envelope. Everything I have said about volume is therefore about this benchmark's payload; the closed form's $p(p-1)n$ term is untested by a sweep in which $n = 1$, and the two algorithms converge on volume as n grows. I have kept the dominance claim to messages and rounds, which are payload-independent.

The per-message-size argument is an inference about a constraint no run exercises. I read the maximum arrival size off the logged `tokens` field per round and extrapolated by multiplying the block count by a hypothetical artefact size. That extrapolation assumes framing stays negligible and that a rank's budget is charged for the whole arrival, which is true when `admit` is set — and the collectives here all pass `admit=False`. So the scenario in which ring wins is constructed from the code's semantics rather than observed, and a run with `strict_context=True` and a large payload would be needed to confirm that Bruck's late rounds actually fail where ring's do not.

Ring's block-index tagging is what makes its message size constant, and a different implementation would not have it. Ring carries `{"idx", "val"}` so that a receiver can file a block it did not compute the position of. A version that relied on the round number to infer the index — which is possible, since in round k the block arriving at rank r originated at $(r - k - 1) \bmod p$ — would send 4-token messages instead of 12 and change the volume comparison above substantially at small payloads. The constancy in p would survive; the factor of 3 against Bruck at $p = 3$ would not. The measurements are of this implementation, not of the algorithm in general.